

Prompts de pruebas adaptados al proyecto YURA

Datos oficiales del proyecto

Título: YURA: Robot interactivo para la practica y seguimiento del aprendizaje del quechua en estudiantes de sexto de primaria de la Unidad Educativa Boliviano Holandes.

Sistema a evaluar: plataforma web YURA Platform, desarrollada con PHP 8.2+, Laravel 12, MySQL/MariaDB, Blade, Vite y Laravel Sanctum.

Proposito: gestionar informacion academica y familiar, registrar notas y asistencia, consultar avances y recibir actividades provenientes del ecosistema del robot YURA.

Repositorio analizado: aplicacion web Laravel disponible en este proyecto.

Componentes confirmados en el repositorio: autenticacion web por sesiones; API protegida con Sanctum; dashboard; usuarios; estudiantes; tutores y familiares; cursos; paralelos; asignaturas; asignaciones docentes; calificaciones; asistencia; historial de calificaciones; actividades del robot; exportaciones PDF y Excel; cambio de idioma; fotografias.

Roles identificados: administrador (`admin`), academico (`academic`), profesor (`profesor`), tutor (`tutor`), padre (`padre`), madre (`madre`), estudiante (`estudiante`) y robot (`robot`), sujetos a verificacion durante las pruebas.

Credenciales de prueba documentadas: `admin@yura.com`, `profesor@yura.com` y `padre@yura.com`, con la contraseña indicada en la guia de instalacion. Deben usarse solo en ambiente de prueba.

Base de datos y entidades observadas: usuarios, estudiantes, cursos, paralelos, asignaturas, asignaciones docentes, tutores/familiares, calificaciones, historial de calificaciones, asistencia, actividades, sesiones de actividad, interacciones, resultados, grupos y tokens de acceso personal. Los nombres y relaciones definitivos deben confirmarse contra las migraciones ejecutadas.

Fuera de evidencia del repositorio: codigo de la APK, software interno del robot fisico, modelo de hardware, procesamiento ASR, formula de precision fonetica, luces, bocinas y actuadores. Esos elementos se registran como pendientes de informacion o pruebas independientes.

Regla documental: no completar resultados obtenidos, estados de ejecucion, defectos o metricas hasta que existan evidencias reales. Cuando falte una regla se indicara `requiere verificacion`.

PROMPT 3 - Plan de pruebas

Actua como responsable de QA y elabora un Plan de Pruebas formal para el proyecto **YURA: Robot interactivo para la practica y seguimiento del aprendizaje del quechua en estudiantes de sexto de primaria de la Unidad Educativa Boliviano Holandes.**

Utiliza unicamente la informacion de este documento y del repositorio descrito. El plan debe cubrir la plataforma web Laravel y su API MySQL/Sanctum.

Incluye obligatoriamente:

- Introduccion.
- Objetivo general y objetivos especificos.
- Alcance: autenticacion, roles, dashboard, usuarios, estudiantes, tutores, familiares, cursos, paralelos, asignaturas, asignaciones docentes, notas, asistencia, historial, actividades del robot, API y exportaciones.
- Fuera de alcance: APK, robot fisico, ASR y hardware, salvo que se entregue evidencia adicional.
- Descripcion del sistema.
- Modulos incluidos.
- Tipos de pruebas: unitarias, funcionales, validacion, integracion, API, autorizacion, persistencia, regresion, seguridad, rendimiento, usabilidad y compatibilidad.
- Estrategia por riesgo, priorizando autenticacion, permisos, datos de menores, notas, asistencia y relaciones.
- Ambiente: XAMPP, PHP 8.2+, Laravel 12, MySQL/MariaDB, navegador actualizado, red local y datos aislados.
- Tecnologias utilizadas: PHP, Laravel, Blade, Vite, MySQL/MariaDB, Sanctum, Pest/PHPUnit, Dompdf y Excel.
- Datos de prueba anonimizados y controlados.
- Roles y responsables.
- Criterios de entrada y salida.
- Riesgos de pruebas, incluyendo inconsistencias `role / rol` , `status / estado` , diferencias web/API, datos de menores y dependencias externas.
- Gestion de defectos con severidad Critica, Alta, Media y Baja.
- Evidencias: capturas, respuestas HTTP, consultas controladas, logs, exportaciones y tiempos medidos.
- Cronograma sugerido.
- Criterios para determinar si esta listo para produccion.
- Conclusiones sin declarar pruebas ejecutadas.

No inventes resultados ni afirmes que una prueba fue ejecutada.

PROMPT 4 - Generacion de casos de prueba

Para el sistema YURA descrito arriba, genera una especificacion profesional y ejecutable manualmente de casos de prueba. Relaciona cada caso con los requisitos RF-001 a RF-008, RNF-001 a RNF-004 y las reglas RN-001 a RN-003 cuando corresponda.

Usa la estructura: ID, requisito relacionado, modulo, nombre, objetivo, prioridad, precondiciones, datos de entrada, pasos, resultado esperado, resultado obtenido, estado, evidencia y observaciones.

Codifica los casos como CP-001, CP-002, etc. Incluye casos positivos, negativos, valores limite, campos obligatorios, datos invalidos, duplicados, operaciones incorrectas, permisos por rol, navegacion y persistencia.

Cubre como minimo autenticacion, usuarios, estudiantes, familiares, cursos, paralelos, asignaturas, asignaciones docentes, notas, asistencia, dashboards, actividades robot, API, exportaciones e historial.

Deja Resultado obtenido vacio, Estado como PENDIENTE y Evidencia como pendiente de adjuntar. No inventes reglas que no consten en la informacion del proyecto.

PROMPT 5 - Pruebas de validacion

Diseña casos de prueba de validacion para los formularios web y endpoints de YURA. Analiza, para cada campo disponible en usuarios, estudiantes, cursos, paralelos, asignaturas, asignaciones docentes, notas, asistencia, familiares y actividades robot:

- obligatorio u opcional;
- tipo de dato;
- longitud minima y maxima;
- formato;
- valores permitidos y no permitidos;
- duplicidad;
- relaciones con otros datos;
- reglas de negocio.

Usa la regla: si una longitud, formato, rango o mensaje no aparece en las migraciones, Form Requests, controladores o reglas visibles, escribir requiere verificacion.

Genera pruebas positivas y negativas para campos vacios, espacios, caracteres especiales, valores cortos y largos, tipo incorrecto, formato incorrecto, duplicados, fuera de rango y combinaciones invalidas. Incluye especialmente notas y asistencia en sus limites, capacidad maxima de paralelos,

tutor principal/secundario, relaciones curso-paralelo y asignaciones duplicadas.

No inventes reglas de validacion.

PROMPT 6 - Pruebas de autenticacion y autorizacion

Disena casos relacionados con autenticacion, autorizacion y control de acceso de YURA.

Roles a verificar: administrador, academico, profesor, tutor, padre, madre, estudiante y robot, confirmando primero cuales estan activos en la aplicacion.

Funcionalidades protegidas: login, logout, dashboard, gestion de usuarios, estudiantes, familiares, cursos, paralelos, asignaturas, asignaciones docentes, notas, asistencia, historial, actividades robot, exportaciones y endpoints API.

Incluye pruebas para login correcto, contrasena incorrecta, usuario inexistente, campos vacios, logout, URL protegida sin autenticacion, funciones no autorizadas, restricciones por rol, acceso posterior al logout, expiracion o manejo de sesion, tokens Sanctum validos/invalidos/revocados y operaciones criticas.

Relaciona cada prueba con un requisito. No declares vulnerabilidades: el resultado debe quedar pendiente hasta ejecutar la prueba. Verifica que un tutor consulte solo estudiantes vinculados y que un profesor opere solo sobre asignaciones autorizadas.

PROMPT 7 - Pruebas de MySQL y persistencia

Disena pruebas para verificar la interaccion Laravel-MySQL/MariaDB de YURA.

Tablas o entidades a considerar: users, estudiantes/students, courses, parallels, subjects, teaching_assignments, tutors/family members, grades, grade_history, attendance, activities, activity_sessions, interactions, results, groups, group_student y personal_access_tokens. Confirma los nombres finales revisando las migraciones.

Verifica insercion, consulta, actualizacion, eliminacion o eliminacion logica, persistencia despues de cerrar sesion, claves primarias, claves foraneas, restricciones, duplicados, datos inexistentes, relaciones entre tablas, integridad referencial y consistencia despues de cada operacion.

Para cada prueba proporciona ID, tabla/modulo, objetivo, precondiciones, datos, pasos, resultado esperado y evidencia necesaria. Incluye relaciones estudiante-paralelo-curso, tutor-estudiante,

profesor-asignacion, calificacion-estudiante/asignatura, asistencia y actividad robot.

No ejecutes ni inventes resultados.

PROMPT 8 - Pruebas de integracion

Identifica y prueba los flujos de integracion internos de YURA entre rutas, controladores, modelos, migraciones, MySQL, formularios, validaciones, middleware, autenticacion, vistas y API Sanctum.

Prioriza estos flujos: login-dashboard; crear curso-paralelo-estudiante; asociar tutor; asignar profesor/asignatura; registrar nota e historial; registrar asistencia; consultar dashboard segun rol; crear actividad robot; actualizar resultado o puntaje mediante API; consultar hijos del tutor; exportar estudiantes a PDF/Excel; logout.

Usa el formato ID, flujo, componentes involucrados, precondiciones, pasos, resultado esperado, resultado obtenido, estado y evidencia.

No inventes componentes. Para APK, ASR y robot fisico indica `no verificable con el repositorio disponible`.

PROMPT 9 - Pruebas de rendimiento

Disena un plan basico de rendimiento para YURA, apropiado para un proyecto academico Laravel/MySQL.

Mide, sin adelantar valores, login, dashboard por rol, listados y busquedas de estudiantes, consultas de notas/asistencia, exportaciones PDF/Excel y endpoints API de dashboard, estudiantes y actividades robot.

Registra tiempo de respuesta, tiempo de carga, usuarios concurrentes, errores HTTP, uso de CPU/RAM, consultas y tiempo de base de datos, tamano de respuesta y consumo de red.

Escenarios: usuario individual, carga normal del aula, concurrencia creciente, consulta con pocos y muchos registros, exportacion de datos y disponibilidad degradada de MySQL. Usa datos anonimizados y volumen documentado.

Herramientas posibles: navegador y DevTools, Pest/PHPUnit para mediciones basicas, Apache JMeter, k6, Laravel Telescope si se habilita, logs del servidor y herramientas de MySQL. Los criterios de aceptacion deben estar definidos por el equipo; si no existen, indicar `requiere verificacion`. Documenta resultados reales en una tabla separada.

PROMPT 10 - Matriz de trazabilidad

Construye una matriz entre los requisitos verificables de YURA y los casos de prueba generados.

Requisitos funcionales: RF-001 captura/inicio, RF-002 transcripcion ASR, RF-003 precision fonetica, RF-004 configuracion docente, RF-005 graficos de avance, RF-006 notificaciones, RF-007 registro de intentos y RF-008 recompensa ludica. Para el repositorio web, separar los requisitos academicos confirmados del componente robotico que requiere evidencia externa.

Requisitos no funcionales: RNF-001 operacion local, RNF-002 usabilidad infantil, RNF-003 latencia de 2.5 segundos y RNF-004 seguridad/privacidad. Reglas: RN-001 correspondencia de curso/nivel, RN-002 puntaje referencial y RN-003 vocabulario configurado.

Presenta: requisito, descripcion, modulo, caso de prueba, tipo, prioridad y estado. Verifica cobertura, requisitos sin prueba, casos sin requisito y requisitos criticos con cobertura insuficiente. Si la relacion no esta sustentada, indicar `requiere verificacion`. Calcula el resumen solo con datos de casos existentes.

PROMPT 11 - Reporte de defectos

Analiza exclusivamente resultados reales de casos de prueba ejecutados en YURA que sean diferentes de lo esperado.

Por cada diferencia confirmada genera: ID, caso relacionado, requisito, modulo, titulo, descripcion, pasos para reproducir, resultado esperado, resultado obtenido, severidad, prioridad, estado, evidencia, causa probable, correccion aplicada, fecha y responsable.

Clasifica severidad como Critica, Alta, Media o Baja. No inventes defectos, causas, correcciones ni fechas. Si no existe evidencia suficiente, indica `informacion insuficiente` y no confirmes el defecto.

PROMPT 12 - Pruebas de regresion

A partir de defectos corregidos y casos originales reales de YURA, disena pruebas de regresion para confirmar la correccion y detectar efectos secundarios.

Cubre nuevamente la funcionalidad corregida y sus relaciones: autenticacion/autorizacion, usuarios, estudiantes, tutores, estructura curso-paralelo, asignaciones, notas, asistencia, historiales, dashboards, actividades robot, API y exportaciones, segun el defecto.

Usa una tabla con ID, defecto relacionado, caso original, caso de regresion, funcionalidad, pasos, resultado esperado, resultado obtenido, estado y evidencia. Mantiene `resultado obtenido` vacio y

estado como PENDIENTE hasta ejecutar.

No agregues regresiones para defectos que no hayan sido documentados.

PROMPT 13 - Informe de resultados

Elabora el Informe Final de Pruebas de YURA usando exclusivamente estos datos reales: total de casos, aprobados, fallidos, pendientes, defectos encontrados, corregidos y pendientes; resultados por caso; defectos; evidencias; y cobertura de requisitos.

Incluye introduccion, objetivo, alcance, ambiente, pruebas realizadas, resumen estadistico, resultados por modulo, defectos, correcciones, regresion, cobertura, evidencias, riesgos pendientes, limitaciones, conclusiones y recomendacion sobre produccion.

Calcula correctamente porcentajes usando el total informado. No conviertas casos pendientes en aprobados, no inventes cobertura y no afirmes ejecucion si no existe resultado. Si faltan datos, declaralo como no informado.

PROMPT 14 - Evaluacion de preparacion para produccion

Actua como evaluador de calidad y analiza si YURA esta listo para pasar de desarrollo a produccion usando solo el informe de pruebas real.

Evalua funcionalidad, validaciones, seguridad, autenticacion, autorizacion, integridad de datos, base de datos, usabilidad, rendimiento, compatibilidad, defectos pendientes, evidencias y cobertura.

Clasifica cada aspecto como APROBADO, APROBADO CON OBSERVACIONES, NO APROBADO o NO EVALUADO.

Determina una sola conclusion: LISTO PARA PRODUCCION, LISTO CON OBSERVACIONES o NO LISTO PARA PRODUCCION. Justifica con evidencia concreta. La APK, el robot fisico, ASR y hardware deben quedar como NO EVALUADO si no se proporciona evidencia de sus pruebas. No supongas resultados ni ausencia de vulnerabilidades.

Checklist de informacion pendiente antes de ejecutar

- Reglas exactas y mensajes de validacion de cada formulario.
- Nombres definitivos de tablas y columnas despues de migrar.

- Datos de prueba anonimizados y volumen esperado.
- Responsables, fechas y ambiente concreto de ejecución.
- Dispositivos y navegadores objetivo.
- Criterios de rendimiento acordados.
- Evidencia de APK, robot, ASR, actuadores y notificaciones si se los va a incluir.
- Requisitos de privacidad y autorización para datos de menores.

Estado del documento: prompts adaptados y listos para completar con resultados reales; ninguna prueba se declara ejecutada.